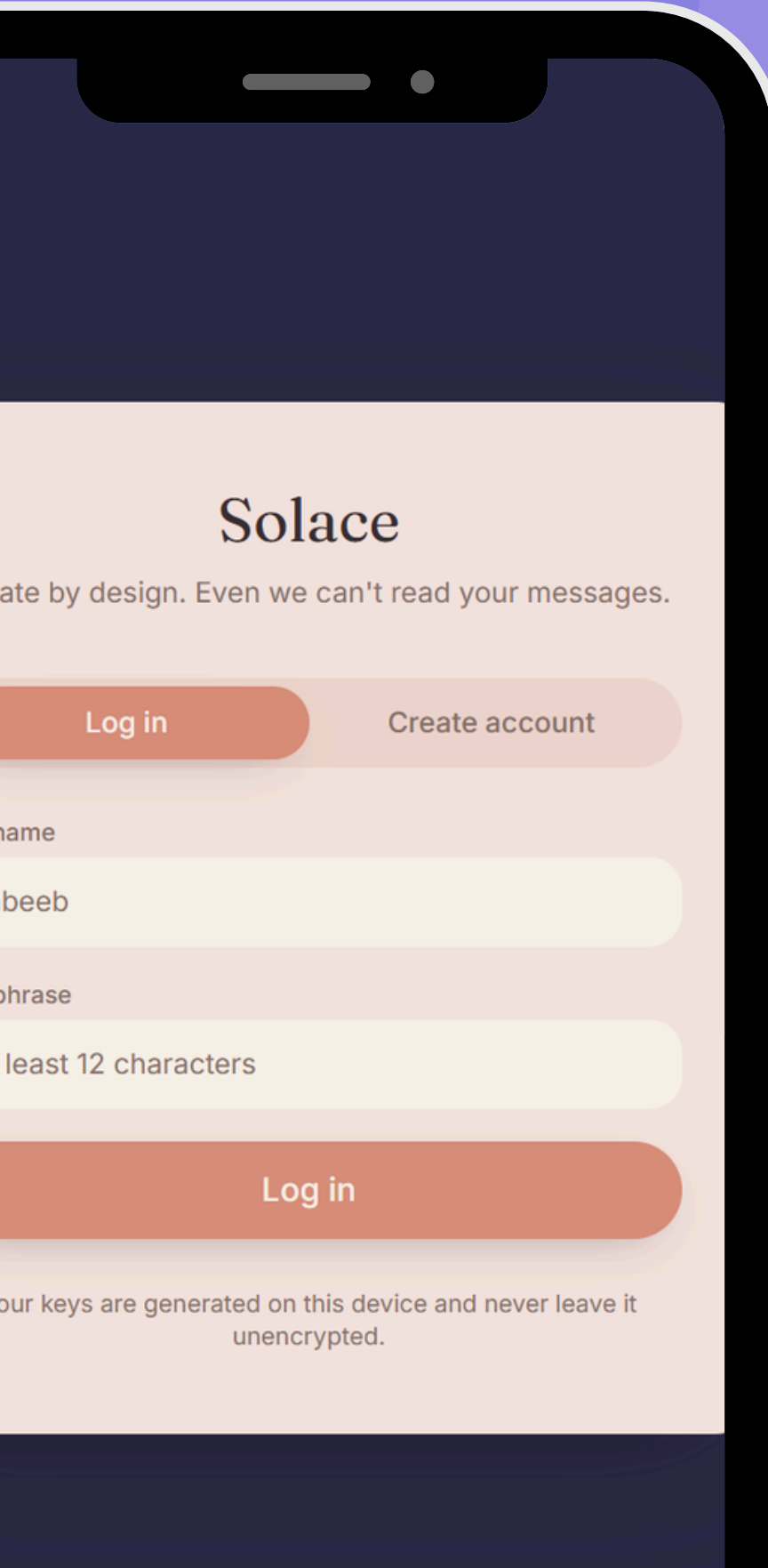




SOLACE

Chat Application with End-to-End Encryption

TEAM MEMBERS:

CHAKRIKA PANAKALA — 2520090042

AMRUTHA JAVVADI — 2520080066

HRUSHIKA HABEEB — 2520090054

Project Focus

What We Focused on in Review 2

Database Design

Organisation of users, conversations, messages and related data

Structured storage and retrieval of application data

Efficient representation of chat information

Data Management

Connecting users with their conversations and messages

Maintaining consistency between related records

Handling message and attachment metadata

Security & Privacy

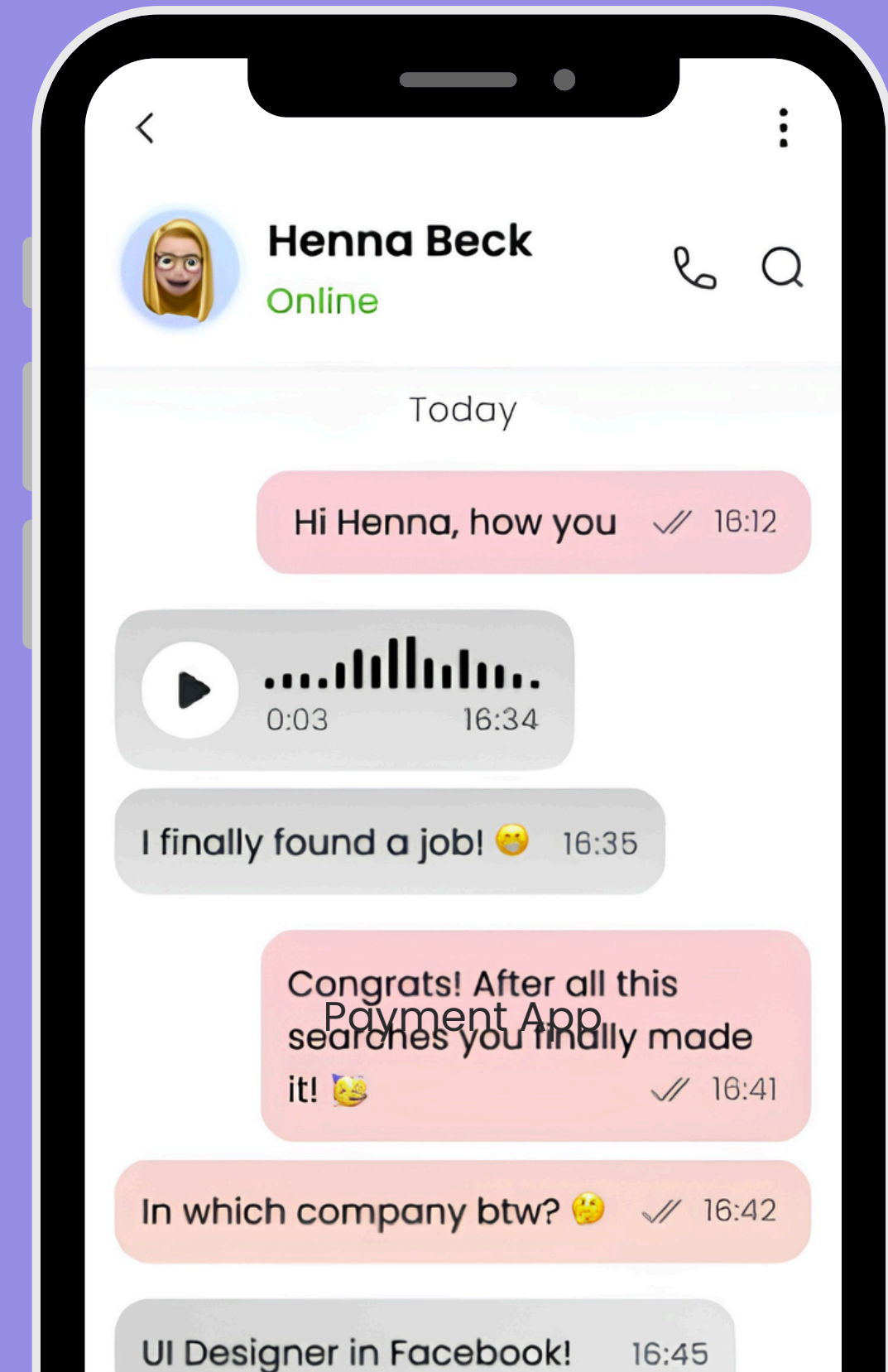
Secure handling of authentication information

Protection of message content

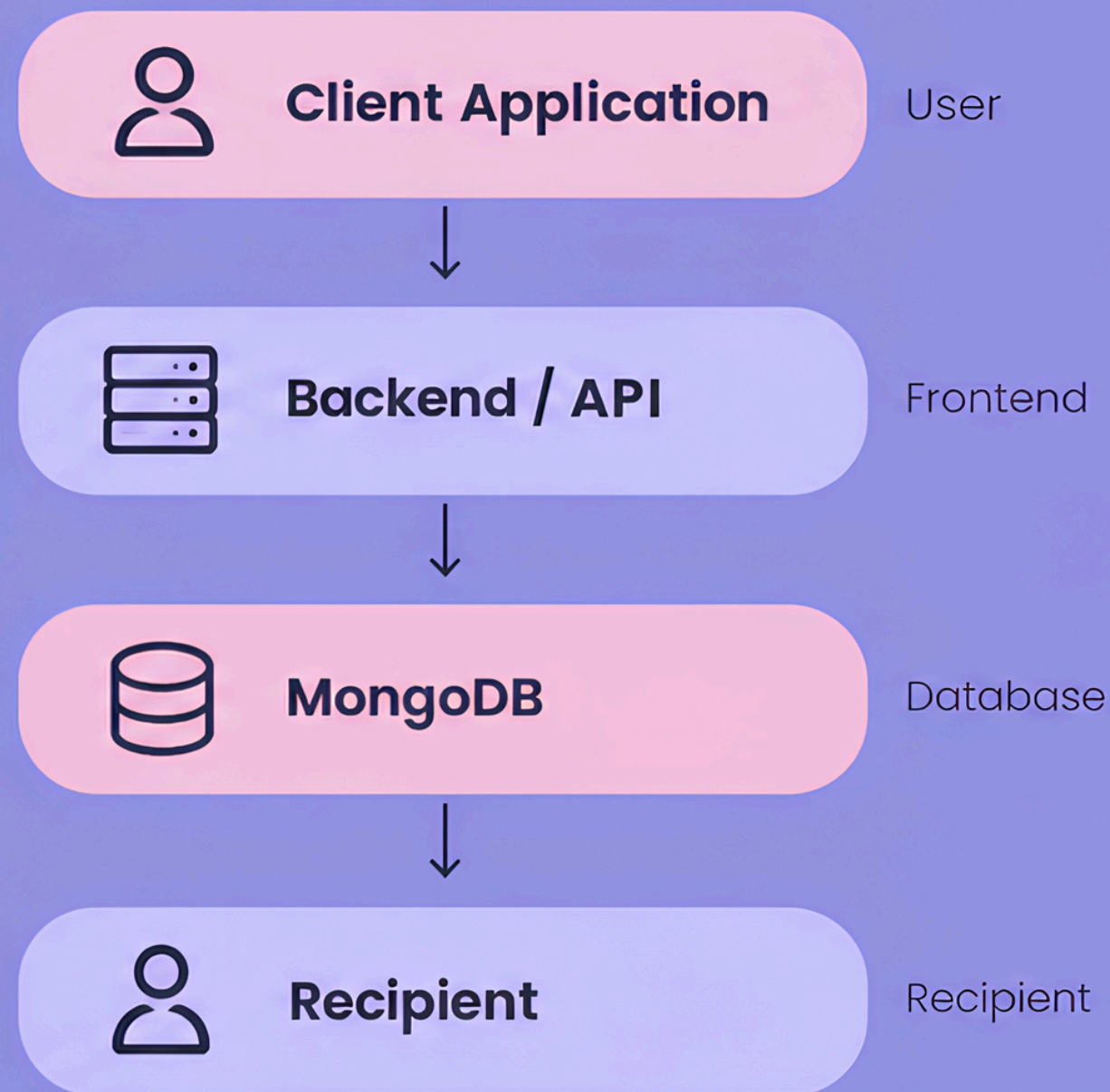
Access control for conversations and user data

Core Objective:

Build a structured backend that supports secure and reliable real-time communication.



System Architecture



User → Frontend → Backend/API → MongoDB → Backend/API → Recipient

1

Client Application

Login, chat interface, message composition and conversation management.

2

Backend / API

Authentication, application logic, request processing and message operations.

3

Database

Users, conversations, messages, media metadata and document references.

4

Recipient

Retrieves authorised conversation data and displays received messages.

Database Design

1 Conceptual Data Model

**User**

Stores account and authentication-related information.

**Conversation**

Represents individual or group chats.

**Message**

Represents individual messages exchanged in a conversation.

**Media**

Represents files or attachments associated with messages.

2 Key Database Concepts

**Identifier**

Uniquely identifies a document or entity.

**Reference**

Connects one document with information belonging to another entity.

**Relationship**

Defines how users, conversations, messages and media are connected.

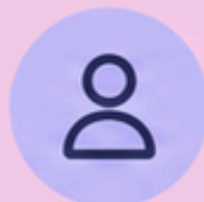
3 Data Relationship

**User****Conversation****Message****Media**

Conceptual model; MongoDB does not use traditional relational foreign-key constraints.

Database Schema

CipherChat stores application data in separate MongoDB collections. Each collection represents a specific type of information and supports efficient data management.



Users

- `_id`
- `username`
- `email`
- `password hash`
- `public-key metadata`



Conversations

- `_id`
- `participant IDs`
- `type`
- `created timestamp`
- `last activity`



Messages

- `_id`
- `sender ID`
- `conversation ID`
- `encrypted content`
- `timestamp`
- `message type`



Media

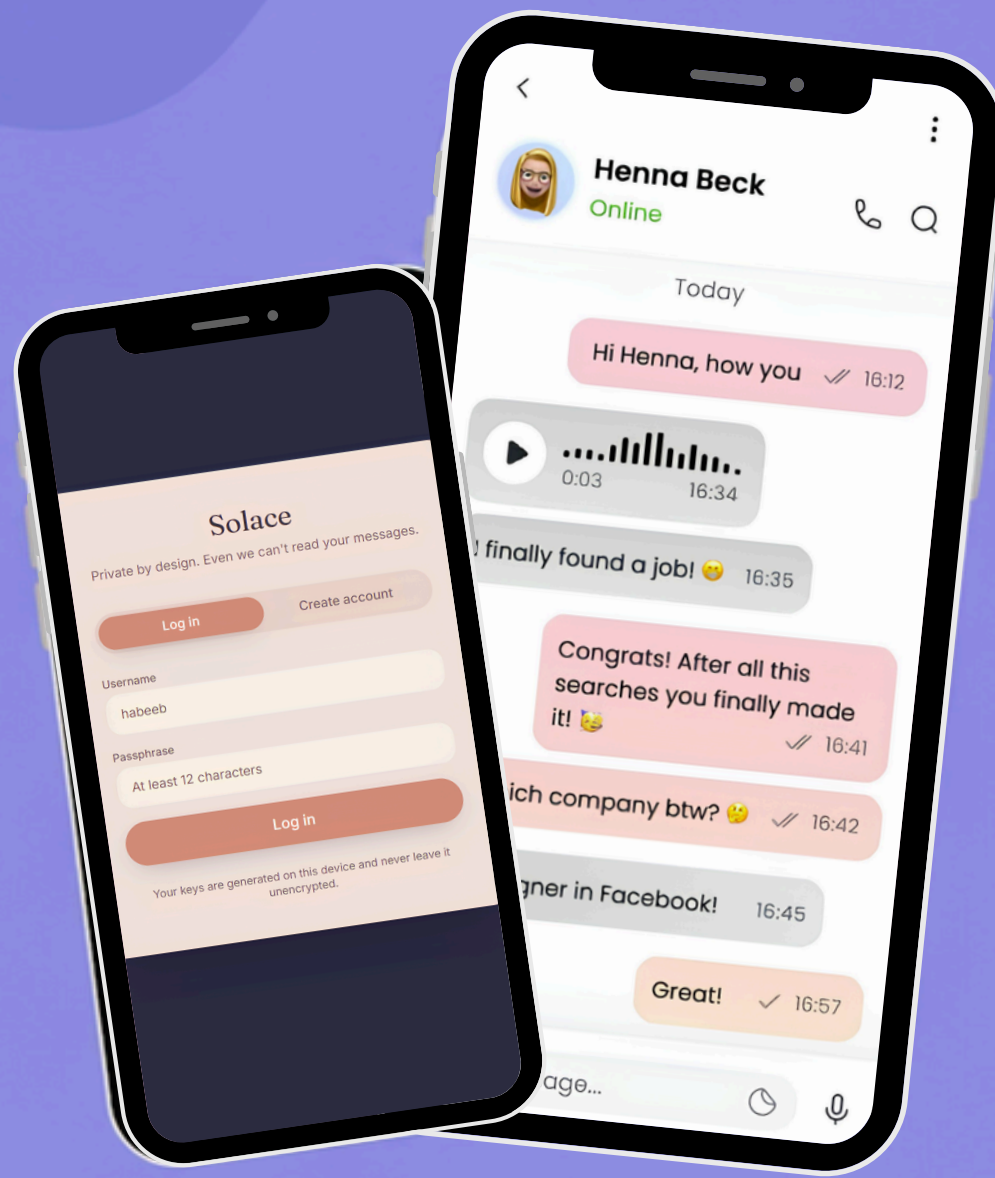
- `_id`
- `message ID`
- `file type`
- `file size`
- `storage info`

Message Life Cycle



Compose → Encrypt → Process → Store → Retrieve → Decrypt → Display

Security & Privacy



Message Protection

- Message content is intended to be protected through encryption.
- Encrypted content should not expose plaintext messages unnecessarily.



Authentication

- User credentials should be handled securely.
- Passwords should never be stored as plain text.
- Use password hashes for password storage.



Access Control

- Users should only access conversations they are authorised to view.
- Backend endpoints should validate user permissions.



Key Management

- Encryption keys must be handled securely.
- Key storage and exchange should be considered separately from message storage.



Security Testing

- Authentication testing
- Access-control testing
- API security testing
- Cryptographic review

Interface ↔ Database

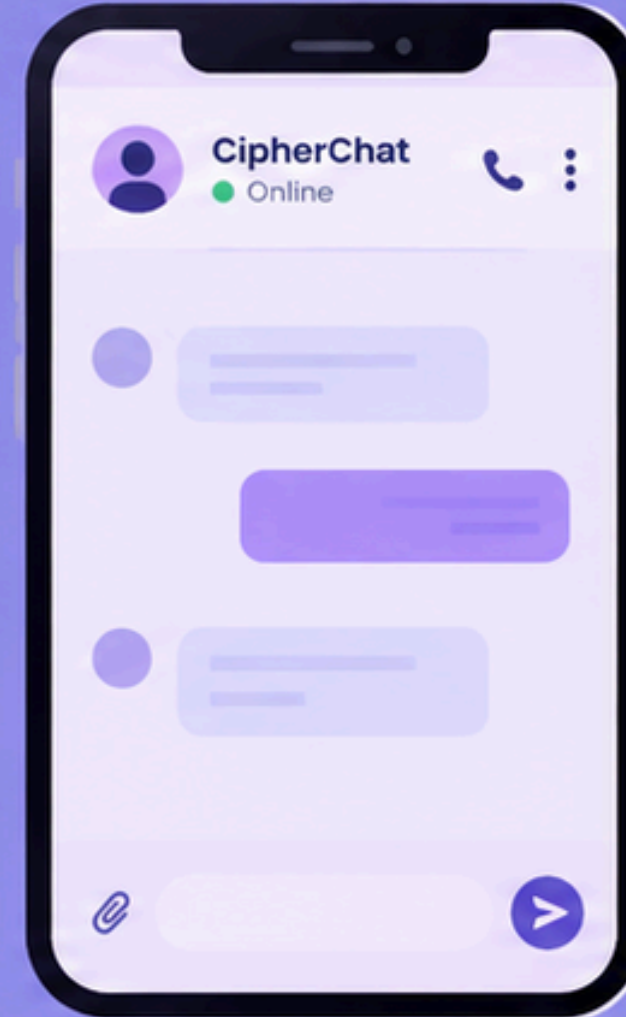
Connecting User Actions with Database Operations

User Action	Backend Operation	Related Data
 Create account	Create user record	Users
 Login	Verify credentials	Users
 Open chat	Retrieve conversation	Conversations
 Load messages	Query conversation messages	Messages
 Send message	Create message record	Messages
 Send attachment	Store attachment metadata	Media
 View conversation	Retrieve authorised records	Users + Messages



Example: Sending a Message

- 1 User enters message
- 2 Frontend sends request
- 3 Backend validates request
- 4 Sender + conversation identified
- 5 Message stored
- 6 Recipient retrieves message



Database Validation & Testing

Ensuring Data Integrity, Security and Proper Functionality



User Validation

- Required fields must be provided
- Email format should be validated
- Duplicate accounts should be handled



Message Validation

- Sender must correspond to a valid user
- Message must belong to a valid conversation
- Required message fields must be present



Conversation Validation

- Participants must be identified correctly
- Users should only access authorised conversations



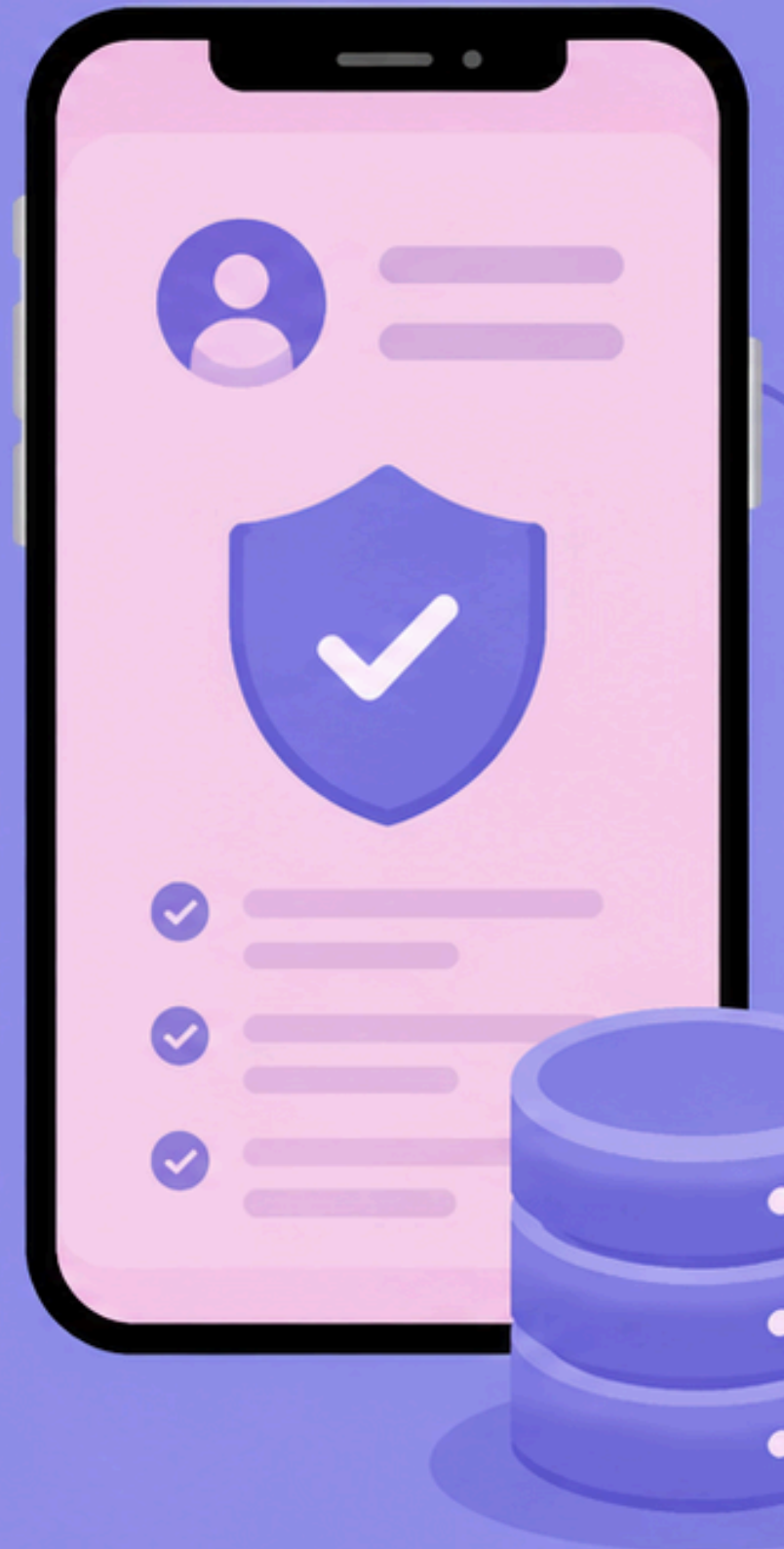
Media Validation

- File type and size should be checked
- Attachment must be associated with the correct message



Security Validation

- Test unauthorised requests
- Test invalid authentication
- Verify that protected message data is handled correctly



Future Scope

Building a Safer, Smarter and More Scalable CipherChat



Security Auditing

Review authentication, authorisation, API endpoints and access-control mechanisms.



Penetration Testing

Test the application against common vulnerabilities, invalid requests and unauthorised access attempts.



Cryptographic Review

Evaluate the encryption algorithm, implementation and key-management approach used by the application.



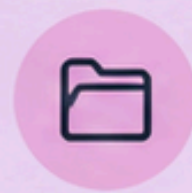
Real-Time Improvements

Improve message delivery, synchronisation and handling of concurrent users.



Database Optimisation

Improve query efficiency, indexing and data retrieval as the number of users and messages increases.



Data Reliability

Strengthen validation, error handling, backup strategies and consistency of stored data.

THANKYOU!